

Gabriel Cárdena López

Programación de sistemas distribuidos

Práctica 4

Ejercicio 1 :

El modelo de practica elegido es el a: ¿Te atreves a hacer la práctica 2 de RMI en otro lenguaje como Python o Ruby? Muestra evidencias del trabajo con capturas.

Primero tuve que familiarizarme con el lenguaje Python ya que solo había programado en él cosas sencillas, en el transcurso de ello descubrí que en Python uno puede levantar un servidor para el servidor al cual el cliente podía conectarse y hacer sus consultas. Entre ellos estaba pyro5 y xmlrpc, acabé utilizando el último ya que me parecía mas correcto para la implementación del programa. Este funcionamiento, que es muy parecido a java rmi, permitía no tener un archivo interfaz en ambos dispositivos, el servidor se encargaba de ello. Traduje el programa de cliente que contiene la "ui", la lógica del programa y la conexión al servidor, también reorganicé y añadí una nueva funcionalidad: potencia.



Después transformé el archivo de servidor el cual inicia un servidor xmlrpc en el puerto 1100 en la ip localhost, en vez de usar las funciones bind() o rebind() utilizo la función register_instance(), además utilizo la función register_function() como método para registrar las funciones que yo he declarado en el servidor, actuando por así decirlo de interfaz por ello los parámetros de entrada de las funciones tienen 3 argumentos self (nombre de la función) y los argumentos de las operaciones. Otro aspecto que recalcar es que como este susodicho archivo ahora no es necesario las funciones declaradas en el servidor no están siendo sobrescritas así que no necesitan @override (ni su equivalente). Un problema que yo he pensado que produce este tipo de arquitectura es que todas las funciones ahora son públicas, si de alguna manera tienes declaradas funciones solo disponibles para el servidor, pero modificas el archivo cliente para poder invocarlas no tendrías problema, he encontrado una solución que es renombrar la función con el símbolo "_" delante.

Declaración e implementación de las funciones.

```
def sumar(self, numero1, numero2):
    return numero1 + numero2

def restar(self, numero1, numero2):
    return numero1 - numero2

def multiplicar(self, numero1, numero2):
    return numero1 * numero2

def dividir(self, numero1, numero2):
    if numero2 == 0:
        raise ValueError("No se puede dividir entre 0")
    return numero1 / numero2

def sqrt(self, numero1):
    if numero1 < 0:
        raise ValueError("No se puede calcular la raíz cuadrada de un número negativo")
    return sqrt(numero1)

def elevar(self, numero1, numero2):
    return pow(numero1, numero2)
```

Inicialización del servidor, conexión del servidor a la calculadora y a las funciones

```
PUERTO = 1100
server = SimpleXMLRPCServer(("localhost", PUERTO))
print(f"Servidor escuchando en el puerto {PUERTO}...")
server.register_instance(Calculadora())
server.register_function(lambda: "Calculadora", "get_name")
server.serve_forever()
```

Funcionamiento del programa tras cambiar la función `eleva()` a privado → `_eleva()`

```
[-1] => Salir
[0] => Sumar
[1] => Restar
[2] => Multiplicar
[3] => Dividir
[4] => Raiz Cuadrada
[5] => Potencia

Elige: 0
Ingresa el número 1: 4
Ingresa el número 2: 5
Resultado => 9.0
Presiona ENTER para continuar

-----

[-1] => Salir
[0] => Sumar
[1] => Restar
[2] => Multiplicar
[3] => Dividir
[4] => Raiz Cuadrada
[5] => Potencia

Elige: 5
Ingresa el número de la base: 2
Ingresa el número exponente: 3
Error: <Fault 1: '<class \'Exception\'>:method "_eleva" is not supported'>
Presiona ENTER para continuar
```

Por ultimo cree un archivo nuevo para los unittest que ahora en vez de hacerlo con la libreria "junit.jupyter" se hace con "unittest". Cree casos de uso para cada función incluso para las excepciones y los comparo con el resultado esperado, además mirando por internet encontré que le puedes poner un rango de valores permitidos, por ejemplo en la raíz cuadrada uno espera un valor pero el real tiene más decimales, utilizando este rango de valores permitidos da como correcto el valor devuelto por el servidor.

```
Dividir(10, 2) => Esperado: 5, Obtenido: 5.0 =>  Correcto
Dividir(1, 3) => Esperado: 0.3333333333333333, Obtenido: 0.3333333333333333 =>  Correcto
Dividir(5, 0) => Esperado: ValueError
.Multiplicar(3, 4) => Esperado: 12, Obtenido: 12 =>  Correcto
Multiplicar(-2, 5) => Esperado: -10, Obtenido: -10 =>  Correcto
.Restar(10, 4) => Esperado: 6, Obtenido: 6 =>  Correcto
Restar(4, 10) => Esperado: -6, Obtenido: -6 =>  Correcto
.Sqrt(16) => Esperado: 4, Obtenido: 4.0 =>  Correcto
Sqrt(2) => Esperado: 1.41421, Obtenido: 1.4142135623730951 =>  Correcto
Sqrt(-9) => Esperado: ValueError
.Sumar(5, 3) => Esperado: 8, Obtenido: 8 =>  Correcto
Sumar(-5, -2) => Esperado: -7, Obtenido: -7 =>  Correcto
.
-----
Ran 5 tests in 0.006s
```

Ejercicio 2:

Diagrama de flujo



